

CLASE 5 - QA Y TESTING POTENCIADO POR GENIA

Informe de Entrega QA Simulador Mundial 2026

Resultado del ejercicio de analisis estatico, ejecucion de tests, SonarQube, Quality Gate y priorizacion RTC de hallazgos para evaluar preparacion hacia produccion.

Proyecto

simulador-mundial

Fecha

18/06/2026

Stack QA

pytest, coverage, Ruff, SonarQube

Decision final

GO con observaciones

1. Resumen Ejecutivo

El proyecto pasa la suite automatizada completa y el analisis SonarQube queda en estado satisfactorio, sin bugs ni vulnerabilidades detectadas. La cobertura supera el umbral minimo recomendado del 80%. La decision es **GO con observaciones** porque quedan code smells y hallazgos de Ruff para limpiar antes de un GO pleno de produccion.

82/82

tests automatizados pasaron

86.5%

cobertura en SonarQube

0

bugs y vulnerabilidades

OK

Quality Gate actual

15

hallazgos Ruff

24

code smells SonarQube

Dashboard SonarQube: <http://localhost:9000/dashboard?id=simulador-mundial>

Resultado SonarQube

Item	Resultado verificado
Proyecto analizado	simulador-mundial
Nombre visible	Simulador Mundial
Estado del analisis	Analysis successful
Quality Gate	OK
URL del dashboard	http://localhost:9000/dashboard?id=simulador-mundial

2. Alcance del Ejercicio

Parte	Actividad	Resultado
Parte 1	Análisis estático con Ruff y priorización RTC	15 hallazgos clasificados por prioridad
Parte 2	SonarQube local y scanner del proyecto	Análisis exitoso para simulador-mundial
Parte 3	Quality Gate con umbrales reales	Quality Gate actual OK; umbral productivo recomendado documentado
Parte 4	Interpretación y priorización con prompt RTC	Plan de fixes priorizado por riesgo

3. Evidencia de Tests

Suite	Archivo	Resultado
QA Clase 5	tests/test_qa_clase5.py	3 passed, 0 failed
E2E / API existente	tests/e2e	10 passed, 0 failed
Suite completa	tests/	82 passed, 0 failed

Casos críticos cubiertos

ID	Caso	Estado
SIM-CHAMP-002	DB vacía: champion coincide con final.winner	Verde
SIM-CHAMP-003	final.winner pertenece a los finalistas	Verde
DASH-CHAMP-001	Dashboard refleja el campeón de la simulación	Verde

Comandos ejecutados

```
.\.venv\Scripts\python.exe -m ruff check .
.\.venv\Scripts\python.exe -m pytest --cov=. --cov-report=xml
.\.venv\Scripts\pysonar.exe --sonar-host-url=http://localhost:9000 --sonar-project-key=simulador-mundial --sonar-project-name="Simulador Mundial" --sonar-python-coverage-report-paths=coverage.xml
```

4. SonarQube y Quality Gate

El analisis de SonarQube fue ejecutado y procesado correctamente para el proyecto simulador-mundial. El resultado final del dashboard queda en **Quality Gate OK**, sin bugs ni vulnerabilidades reportadas.

Metrica	Resultado	Evaluacion
Quality Gate actual	OK	Cumple
Coverage	86.5%	Cumple umbral >= 80%
Bugs	0	Cumple
Vulnerabilities	0	Cumple
Duplicated lines density	0.0%	Cumple
Code smells	24	Observacion
NCLOC	767	Informativo

Quality Gate recomendado para produccion

Metrica	Umbral
Bugs	0
Vulnerabilities	0
Coverage	>= 80%
Duplicated lines density	<= 3%
Code smells	<= 20
Tests automatizados	100% passing

Nota: el token usado permite consultar SonarQube, pero no crear/asignar Quality Gates desde API (actions.create=false). Por eso el Quality Gate productivo queda documentado como criterio recomendado de evaluacion.

5. Smoke Test Frontend

Estado general: **GO con observaciones**. La suite automatizada valida carga del frontend por HTML, simulacion, dashboard, bracket y consistencia del campeon. Quedan pendientes validaciones visuales/manuales: spinner, click real del boton y responsive mobile en navegador.

ID	Validacion	Estado
FE-SMOKE-001	La app carga y se ve #teamsContainer	Paso automatizado
FE-SMOKE-002	Boton #btnSimular dispara simulacion	Paso parcial
FE-SMOKE-003	Spinner aparece y desaparece	Pendiente visual
FE-SMOKE-005	Campeon visible en #championResult	Paso parcial
FE-SMOKE-006	Dashboard muestra metricas	Paso automatizado
FE-SMOKE-007	Bracket muestra eliminatorias y ganadores	Paso automatizado
FE-SMOKE-008	Usabilidad mobile 375px	Pendiente visual

6. Priorizacion RTC de Hallazgos

Prioridad	Hallazgo	Riesgo	Accion propuesta
P1	services/simulator_service.py: variable y sin uso en validacion	Puede ocultar logica incompleta sobre jugadores requeridos	Eliminar la variable o reemplazarla por validacion explicita si corresponde a la regla de negocio
P1	SonarQube: 24 code smells vs umbral recomendado <= 20	Deuda tecnica superior al umbral productivo	Atacar primero smells vinculados a simulacion y servicios
P2	Imports sin uso en servicios y tests	Ruido y menor mantenibilidad	Remover imports con ruff check . --fix y revisar cambios
P2	schemas/team.py: import a mitad del modulo	Orden de imports fragil; posible dependencia circular	Reordenar schema o aislar el model_rebuild()
P3	f-strings sin placeholders	Estilo innecesario, sin impacto funcional	Quitar prefijo f

7. Conclusion

La conclusion del ejercicio es **GO con observaciones**. El proyecto demuestra buen nivel de confianza funcional: los tests automatizados pasan, la regla critica `champion = final.winner` queda cubierta, SonarQube no reporta bugs ni vulnerabilidades y la cobertura supera el 80%.

Para pasar a GO pleno de produccion se recomienda limpiar Ruff, bajar los code smells a 20 o menos, y completar las validaciones visuales manuales del frontend en navegador real.

Evidencia complementaria disponible en el proyecto: `QA_REPORT.md`, `SMOKE_TEST_REPORT.md`, `allure-report.html` y dashboard local de SonarQube.